

EMD51805-DESIGN PROJECT-IV

PERSONAL ASSISTIVE ROBOT

Submitted By

Shaik Sameer Salahuddin	22125004
Thirisha.R	22125022
Adam Hamdullah Muneer Vm	22125028

Under The Guidance

Of

Guide: Dr S Janaki Raman

BACHELOR OF TECHNOLOGY

IN

MECHATRONICS



HINDUSTAN
INSTITUTE OF TECHNOLOGY & SCIENCE
(DEEMED TO BE UNIVERSITY)

DEPARTMENT OF MECHATRONICS ENGINEERING
HINDUSTAN INSTITUTE OF TECHNOLOGY AND SCIENCE

APRIL 2025



HINDUSTAN

INSTITUTE OF TECHNOLOGY & SCIENCE
(DEEMED TO BE UNIVERSITY)

BONAFIDE CERTIFICATE

Certified that this project report titled “**PERSONAL ASSISTIVE ROBOT**” is the Bonafide work of **Shaik Sameer Salahuddin (22125004), Thirisha. R (22125022), Adam Hamdullah Muneer Vm (22125028)** who did the task under my supervision during the academic year 2024 – 2025.

HEAD OF THE DEPARTMENT

Dr. S Janaki Raman

Associate Professor and Head i/c

Department of Mechatronics Engineering

Hindustan Institute of Technology and Science

Padur, Chennai-603103

SUPERVISOR

Dr. S Janaki Raman

Associate Professor and Head i/c

Department of Mechatronics Engineering

Hindustan Institute of Technology and Science

Padur, Chennai-603103

INTERNAL EXAMINER

Name: _____

Designation: _____

Project Viva-voce conducted on: _____

EXTERNAL EXAMINER

Name: _____

Designation: _____

ABSTRACT

This project provides a comprehensive comparative analysis of two widely used path-planning algorithms: A* (A-star) and RRT (Rapidly-exploring Random Tree), focusing on their applications in autonomous drone navigation. The study involves implementing both algorithms in MATLAB to create 3D path-planning simulations within a dynamically structured environment filled with obstacles, similar to those encountered in real-world search and rescue or inspection scenarios.

A* is a heuristic-based search algorithm known for finding optimal paths by expanding nodes systematically, while RRT is a sampling-based method designed for the rapid exploration of high-dimensional spaces. Each algorithm's performance was evaluated based on key metrics, including path efficiency (measured by path length and smoothness), computational complexity (execution time and memory usage), and obstacle-handling capabilities. The findings highlight significant contrasts in the suitability of each approach: A* excels in structured, grid-based environments with fewer dimensional constraints, providing optimal paths, while RRT is better suited to complex, high-dimensional spaces where rapid, feasible path generation is prioritized over strict optimality.

This analysis offers valuable insights into the strengths and limitations of A* and RRT for diverse autonomous navigation tasks, providing a foundation for selecting appropriate path-planning strategies in specific operational contexts.

Keywords: A*, RRT

ACKNOWLEDGEMENT

First and foremost, we would like to thank the Lord Almighty for his presence and immense blessings throughout the project work. It's a matter of pride and privilege to express my deep gratitude to the HITS management for providing me with the necessary facilities and support.

We are highly elated to express my sincere and abundant respect to the Vice-Chancellor , **Dr. S. N. Sridhara**, for giving me this opportunity to present and implement my ideas in this project. We wish to express my heartfelt gratitude to **Dr. S Janaki Raman**, Associate Professor and HoD i/c, Department of Mechatronics Engineering, for his valuable support and encouragement in carrying out this work.

We would like to thank my guide, **Dr. S Janaki Raman**, Associate Professor of the Department of Mechatronics Engineering for continually guiding and mentoring us by giving valuable suggestions to complete the project work.

We are exceptionally obligated to our project coordinators, **Dr. S Janaki Raman** and **Dr.Sivakamasudari** for their direction and steady supervision throughout our project. We would like to thank all the technical and teaching staff, and the Department of Mechatronics Engineering for all their support.

Last, but not least, my thanks and thanks additionally go to my partner in building up the venture and individuals who have energetically gotten me out with their capacities. We are deeply indebted to my parents who have been the greatest support throughout the project.

Shaik Sameer Salahuddin	22125004
Thirisha.R	22125022
Adam Hamdullah Muneer Vm	22125028



HINDUSTAN
INSTITUTE OF TECHNOLOGY & SCIENCE
(DEEMED TO BE UNIVERSITY)

DEPARTMENT OF MECHATRONICS ENGINEER

EMD51512-AI&ROBOTICS

Project Batch No : 10

Title of The Project : PERSONAL ASSISTIVE ROBOT

Number of The Students In The Batch : 3

Student's Individual Contribution

Name of the Student: Shaik Sameer Salahuddin

Register No.:22125004

Contribution : Developed and implemented A* For the robot, Voice recognition module programming, power circuit design, Robotic arm kinematics, Python programming

I developed and implemented the A* algorithm for the robot, programmed the voice recognition module, and worked on power circuit design. I also handled robotic arm kinematics and contributed to system integration through Python programming.

Shaik Sameer Salahuddin (22125004)



HINDUSTAN
INSTITUTE OF TECHNOLOGY & SCIENCE
(DEEMED TO BE UNIVERSITY)

DEPARTMENT OF MECHATRONICS ENGINEERING

EMD51512-AI&ROBOTICS

Project Batch No : 10

Title of the project : PERSONAL ASSISTIVE ROBOT

Number of the students in the batch : 3

Student's Individual Contribution

Name of the Student:Thirisha. R

Register No.:22125022

Contribution : Object detection and face recognition using YOLO11n, CAD modelling of the Robot, Robotic arm kinematics, Components selection, Hardware integration, Raspberry pi programming

I have implemented object detection and face recognition using YOLOv11n, designed the robot's CAD model, and developed robotic arm kinematics. I have also handled component selection, hardware integration, and Raspberry Pi programming to ensure seamless system functionality.

Thirisha.R (22125022)



HINDUSTAN
INSTITUTE OF TECHNOLOGY & SCIENCE
(DEEMED TO BE UNIVERSITY)

DEPARTMENT OF MECHATRONICS ENGINEERING

EMD51512-AI&ROBOTICS

Project Batch No : 10

Title of the project : PERSONAL ASSISTIVE ROBOT

Number of the students in the batch : 3

Student's Individual Contribution

Name of the Student: Adam Hamdullah Muneer Vm

Register No.: 22125028

Contribution : Voice recognition module training , Components selection, Components Setup, Hardware integration,

I trained the voice recognition module, carried out component selection and setup, and contributed to hardware integration.

Adam Hamdullah Muneer Vm(22125028)

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
-	Abstract	i
-	Acknowledgment	ii
-	Individual Contribution	iv
-	List of Figures	vii
-	List of Tables	viii
1	INTRODUCTION	1
2	LITERATURE REVIEW	
2.1	Introduction	3
2.2	Inference from Research Papers	4
2.3	Summary and Research Gap	10
2.4	Research Gap	10
3	METHODOLOGY	
3.1	Design and Parameter	12
3.2	Components Selection	15
3.3	CAD Modelling	24
3.4	Algorithm Selection	25
4	PROGRAMMING	29
5	RESULTS	36
6	CONCLUSION	42
7	REFERENCES	44
8	PLAGIARISM REPORT	46

LIST OF FIGURES

FIGURE NO.	DESCRIPTION	PAGE NO.
1	Force and Torque Calculator	21
2	Forward Kinematics	22
3	Inverse Kinematics	23
4	Raspberry Pi 5	24
5	Raspberry Pi Camera Noir Module 3	24
6	AI Think Voice Recognition Module	24
7	Ultrasonic Sensor	24
8	Robot CAD model (Front View)	25
9	Robot CAD model (Side View)	25
10	DXF file of Robot	26
11	Charging Station STL File	26
12	A* Algorithm in NumPy	27
13	Gripper 3D Printed	36
14	Charging Station 3D Printed	36
15	Acrylic Sheets	36
16	Fully Assembled Robot	36
17	Robotic Arm	36
18	Object Detection	36
19	Face Detection	36

LIST OF TABLES

TABLE NO.	DESCRIPTION	PAGE NO.
1	Torque and RPM Requirement Summary	15
2	Robotic Arm Joint Specification Table	16
3	Components Used in Robot Construction	23

CHAPTER 1

INTRODUCTION

Over the past few decades, robotics and artificial intelligence have evolved at a pace that borders on the miraculous. Where they were once limited to the factory line and state-of-the-art research facilities, robots today are gradually making their presence felt in our homes, schools, hospitals, and public places. This is primarily because of improvements in low-cost computing capabilities, better sensors, open-source programming, and the growing need for smart systems that are capable of assisting and complementing human routines. With these developments, the concept of a personal robot assistant is no longer fiction—it is becoming a realistic and practical possibility.

This project investigates the design and implementation of a Personal Assistive Robot (PAR)—a general-purpose, interactive robot that is intended to assist users in performing simple everyday tasks. In contrast to specialized assistive devices developed for older adults or individuals with disabilities, this robot is designed with inclusivity. It is not aimed at a particular user category but is rather conceived as an adaptable and approachable companion that can benefit anybody. Whether being used to assist in picking up items that have been dropped, to move small objects from one location to another, or just for the purpose of casual interaction with voice and speech, the PAR has the goal of simplifying life, making it more efficient, and more enjoyable.

The overall objective of this project is to integrate multiple cutting-edge technologies such as computer vision, speech recognition, motor control, and smart decision-making into a small, portable package that can run independently in indoor spaces. Fundamentally, the robot runs on a Raspberry Pi 5, a small and affordable computing module that offers adequate processing power to handle an assortment of sensors, control programs, and communication protocols. The Raspberry Pi 5 acts as the robot's brain, managing everything from real-time image analysis to voice command interpretation and mechanical movement of its arms and wheels.

In order to make the robot see and perceive the world around it, a NOIR (No Infrared Filter) Pi Camera is attached to the robot. This camera takes live visual data and processes it through YOLOv11n, a simple yet powerful object detection algorithm. YOLOv11n (You Only Look Once version 11n) has been recognized for finding a good balance between speed and accuracy, hence its usability in edge devices such as the Raspberry Pi. Using this technology, the robot can recognize and detect different objects and human faces in its surroundings. Object detection is essential in order to allow the robot to

engage with its environment in an intelligent way—for instance, recognizing a bottle on the ground and how to grasp it, or recognizing a particular person to begin a customized interaction.

For human-robot interaction, voice control is used as a natural interface. With the use of a microphone (most likely accessed through a mobile app or physically connected to the Raspberry Pi), the robot is able to hear voice commands. The voice inputs are interpreted through a speech-to-text engine—like Google's Speech Recognition API—which transcribes the audio signals into functional text commands. The robot acts on these commands, performs the respective function, and gives a response through a text-to-speech (TTS) system, providing a conversational interface to the user. This bidirectional voice communication allows a more intuitive and user-centered interaction paradigm, eliminating the necessity of physical input interfaces such as keyboards or touch screens.

Robotically, the robot is capable of mobility and manipulation. It has two motorized wheels that enable it to move around indoor environments, such as carpeted rooms and flat areas. The mobility system consists of a basic differential drive mechanism that can be programmed for path planning and obstacle avoidance. To the left and right of the robot are two 5-DOF robotic arms. These arms have human-like motion and can reach, grasp, lift, and set down small items with fair accuracy. There is a gripper mechanism at the tip of each arm for picking up things firmly so that the robot may engage with objects in its world in important ways.

This integration of vision, voice, motion, and manipulation makes the Personal Assistive Robot a rich source for studying real-world robotics. It is not merely a technical endeavor but also a human-centered one—purposed to improve the way humans engage with machines in the home environment. By combining AI and hardware into an easy-to-use, portable system, this robot shows what can be accomplished through intelligent technology integrating into daily life.

In the pages that follow, we go into greater detail about the technical aspects that give the robot life. We discuss the hardware setup in depth, from sensor choice to mechanical design. We look at the software architecture, such as the AI models, decision-making logic, and integration methods that enable the robot to function independently. We also discuss specific tasks the robot can accomplish, and the challenges encountered during development—from hardware constraints to data processing hurdles. Lastly, we project forward to possible future enhancements and increased use, and speculate how this prototype might develop into an even more powerful and accessible personal robotic aide.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction:

The integration of robotics into daily human life is rapidly evolving, with a strong focus on creating systems that go beyond mechanical assistance to provide meaningful social and emotional interaction. As robots transition from industrial tools to companions and assistants in domestic, healthcare, and collaborative work environments, the need for socially intelligent and emotionally responsive systems becomes increasingly critical. Socially Interactive Robots (SIRs) and assistive robots are designed to bridge this gap by mimicking human communication cues and adapting to user behavior, thereby enhancing usability, trust, and long-term acceptance.

Recent advancements in artificial intelligence, cognitive science, and human-robot interaction have laid the groundwork for robots that not only perform tasks efficiently but also understand and respond to human emotions, intentions, and social contexts. These developments hold promise for a wide range of applications, from eldercare and disability support to personalized healthcare and industrial collaboration. However, despite the promising outcomes, challenges remain in achieving natural, context-aware interaction and sustained user engagement over time.

This chapter explores existing research in socially interactive and assistive robotics, providing a literature-based foundation to understand the current state of the field. It examines various approaches to emotional modeling, interaction modalities, intention recognition, and design principles that shape human-robot relationships. Through a critical review and synthesis of prior work, the chapter also identifies existing limitations and uncovers gaps in the research that must be addressed to realize the vision of truly empathetic and adaptive robotic systems.

2.2. Literature Review

Ishiguro et al. [1] The paper describes the creation of "Robovie," an interactive humanoid robot that is intended to engage in meaningful social interaction and form long-term relationships with people. The architecture of the robot was developed based on cognitive experiment discoveries, where the patterns of human interaction and emotional signals were studied. Robovie is equipped with multimodal communication functions, including speech, gestures, and control of gaze, to facilitate natural interaction. The researchers highlight that the integration of cognitive science and robotics can result in emotionally intelligent and socially competent robots. Inference :Integrating cognitive models into robots increases their social presence and acceptability. The study highlights the need for

interdisciplinary research to develop robots that are capable of perceiving and responding to human emotions.

Fong et al. [2] This survey article presents an extensive overview of socially interactive robots (SIRs), setting the stage for how such systems are designed and assessed. It presents a comprehensive taxonomy with robot roles (e.g., peer, assistant, mediator), interaction modalities (verbal, non-verbal), and system components (sensors, actuators, control architecture). Also, it evaluates the psychological and societal costs of using such systems in public and private environments. Inference :The taxonomy provides a platform for future studies by classifying the intricacies of SIR design. The article highlights the social and emotional aspects of human-robot interaction, extending the frontiers beyond functional performance.

Ratsamee et al. [3] This article presents the design and deployment of a "Robot Companion" customized for Industry 5.0, emphasizing human intention recognition and cooperative task achievement. Through human path prediction and face orientation, the robot predicts human movement to support industrial applications. It demonstrates the combination of vision systems, machine learning, and embedded control for dynamic assembly line cooperation. Inference :The research confirms that robots with the ability to interpret human intention can greatly improve safety and productivity in collaborative workspaces. Human-aware navigation and gesture-based interaction are crucial for seamless cooperation in industrial environments.

Mišeikis et al. [4] This paper presents "Lio," a service robot designed for caregiving and social support within healthcare facilities. Lio comes with a suite of sensors in the form of LiDAR, cameras, and haptic feedback systems, along with a robotic arm to manipulate objects and engage with patients. Its configuration focuses on protecting privacy, patient engagement, and low user cognitive load. Lio's autonomous and semi-autonomous capabilities vary from logistical assistance (e.g., delivery of items) to patient interaction through dialog and reminders. Inference :Bots such as Lio illustrate how assistive technologies can fill staffing gaps in the healthcare sector. Easy-to-use design and context-aware autonomy are crucial for real-world deployment in sensitive settings such as hospitals and care homes.

Mariagrazia Costanzo et al. [5] This article discusses the existing status of AI-based assistive technologies for the elderly, such as smart home systems, wearable health monitors, and interactive robots. The review emphasizes the psychological acceptance of these technologies, particularly their perceived usefulness, ease of use, and emotional impact. It discusses the ways in which socio-cultural considerations and caregiver engagement affect the effectiveness of such systems. Inference :Though

technology is essential to eldercare, human considerations are central. It does not merely rely on functionality but whether it is done with empathy and respect as it is incorporated into daily life.

Duerstock et al. [6] This paper assesses assistive robot deployment for individuals with disabilities, analyzing the role of mobile platforms in supporting daily activity, environmental navigation, and communication. Emphasis is on improving autonomy and lowering caregiver burden. Robots for this purpose need to cope with varying environments and engage using accessible user interfaces like voice and adaptive touchscreens. Inference :For individuals with disabilities, robots should be intuitive, extremely reliable, and customizable. The research identifies the potential for robotics to enhance autonomy and inclusion of differently-abled people, as long as the systems are designed ethically and contextually aware.

2.4 Research Gap:

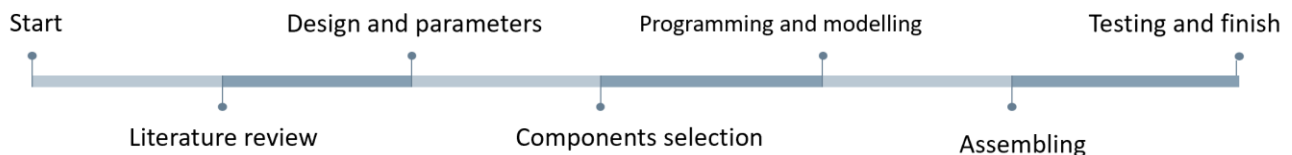
While current advancements showcase impressive progress in robot capabilities and social intelligence, there remains a noticeable gap in the seamless integration of emotional awareness, adaptive learning, and context-specific behaviour across diverse real-world settings. Most systems are domain-specific and lack generalizability, limiting their broader applicability. Additionally, long-term user adaptation, ethical deployment, and cultural sensitivity are underexplored areas, calling for interdisciplinary research that combines AI, human psychology, and social sciences to develop universally acceptable, emotionally responsive robotic companions.

CHAPTER 3

METHODOLOGY

The literature review was fully carried out, and it led to the identification of a number of key parameters for the robot. They include a weight capacity of 5kg so that the robot is not too heavy and cannot be easily manoeuvred. The lifting weight capacity was established at 2–3kg, which is enough to lift numerous objects and perform most tasks. The addition of a 5DOF (Degrees of Freedom) robotic arm enables diverse and accurate movements, making the robot more functional. The motor is at 100RPM (Revolutions Per Minute) with a torque of 98.1, making it perform smoothly and powerfully. Lastly, the robot comes with wheels that are 100mm in diameter, which add to its stability and mobility on various surfaces. These well-thought-out specifications guarantee that the robot is not only effective but can effectively do what it is meant to do.

Next in the process of development will be the construction of a Computer-Aided Design (CAD) model of the robot to offer a detailed visual insight and reveal possible design flaws. After CAD modeling, laser cutting technology will be employed to create most of the structural parts of the robot from acrylic sheets, providing accuracy and strength. 3D printing will be used specifically for the gripper and charging station, where more sophisticated and tailored geometries are needed. After preparing the components, suitable parts and materials will be chosen to maximize performance. Programming of the robot functions will then be done, including AI algorithms, voice interaction features, and object detection mechanisms. Ultimately, the robot's performance will undergo strict testing in order to determine if it performs as expected and needs adjustments or improvements to realize the desired degree of efficiency and reliability. Under this thorough development process, the robot will be perfected and set for practical deployment



3.1 Design and Parameter

The movement of the robot is obtained by precise movements via servo motors. The base provides 360° yaw rotation via an MG995 servo, and the elbow joint, driven by an MG995, provides limited pitch (up/down) movement. The wrist joint, driven by an SG90 servo, offers 360° roll rotation, and the gripper, driven by another SG90, offers an open/close movement. The mechanical framework is constructed completely from laser-cut acrylic sheets, which are carefully designed and combined to hold each joint and component in place. The base platform is constructed from durable acrylic layers to accommodate the yaw motor securely. The shoulder and elbow joints are constructed with interlocking acrylic frames that mimic typical robotic arm angles and hold the servos. The upper arm is made of a 15 cm long, 50 mm wide acrylic segment, and the wrist joint is attached to a compact acrylic bracket for stability. The lower arm is made of a 5 cm long, 32 mm wide acrylic piece, and it gives a compact and light framework to the gripper. The servos employed are the SG90 with a torque of 1.8 kg.cm and the MG995 with a torque of 10 kg.cm, which provide sufficient strength for smooth and controlled movement.

The charging station and robot power system comprise basic elements for efficient power delivery and battery management. The charging station comes with a 12V-5A power adapter, DC-DC step-down voltage regulator (XL4015 with voltmeter), and copper tape or strip for contact charging. Schottky diodes (10A, 1N5822) are also incorporated to protect against reverse current, a 10A protection fuse, and a 12V charging LED indicator with a resistor. A DC barrel jack connector, chosen according to the adapter, provides a secure power connection.

For the robot's charging and power connections of the robot, a 3S BMS module (12.6V, 25A) for 18650 manages charging and discharging. A LiPo battery powers the system, with the connectors like JST or XT60 used in secure wiring. Epoxy glue or a hot glue gun is used to provide hard mounting and insulation.

The robot also uses a Raspberry Pi 5 as its main controller, with a Raspberry Pi Camera Module 3 for computer vision applications. An AI Think Voice Recognition Module allows voice control functionality, with an ultrasonic sensor helping with distance measurement. Motor movement is controlled by the L298N motor driver, multiple servos are efficiently driven by a 16-bit servo driver, and 200 RPM motors offer movement and speed to the robot.

Main functionality ;

- Carry out pick-and-place operations with its gripper.
- Turn and position objects with wrist and elbow joints.
- Move through spaces with obstacle detection.
- Communicate with users through voice commands and camera-based recognition.
- Recharge automatically through a contact-based smart charging dock.

3.2 COMPONENTS SELECTION

3.2.1 MOTOR AND WHEELS SELECTION

To determine the torque and RPM required for a 10 kg robot designed to lift an additional 2–3 kg, several factors need to be considered, including the type of movement (e.g., wheeled or legged), the desired speed, and the lifting mechanism's design. Assuming a wheeled robot operating on flat terrain, each motor must generate enough torque to overcome the robot's weight and any additional load while maintaining balance and control.

Torque and RPM Calculation for Robot

Parameters: (assumed)

1. **Total Weight:** 10 kg
2. **Acceleration:** 0.5 m/s²
3. **Desired Speed:** 1 m/s
4. **Wheel Diameters:** 70 mm and 100 mm

Formulas Used:

1. **Force (F)** = Mass (m) × Acceleration (a)

$$F = m \times a \quad F = m \times a$$

2. **Torque (τ)** = Radius (r) × Force (F)

$$\tau = r \times F \quad \tau = r \times F$$

$$\text{Angular Velocity } (\omega) = \frac{\text{Desired Speed } (v)}{\text{Radius } (r)} \quad \omega = \frac{v}{r}$$

$$3. \text{ RPM} = \frac{\omega \times 60}{2\pi}$$

Calculations:

For 70 mm Wheel Diameter:

- **Radius (r)** = 70 mm / 2 = 35 mm = 0.035 m
- **Force (F)** = 10 kg × 0.5 m/s² = 5 N
- **Torque (τ)** = 0.035 m × 5 N = **0.175 Nm**
- **Angular Velocity (ω)** = 1 m/s ÷ 0.035 m = 28.57 rad/s
- **RPM** =
$$28.57 \times 60 \times \frac{2\pi}{2\pi} = 272.84 \text{ RPM}$$

For 100 mm Wheel Diameter:

- **Radius (r)** = 100 mm / 2 = 50 mm = 0.05 m
- **Force (F)** = 10 kg × 0.5 m/s² = 5 N
- **Torque (τ)** = 0.05 m × 5 N = **0.25 Nm**
- **Angular Velocity (ω)** = 1 m/s ÷ 0.05 m = 20 rad/s
- **RPM** =
$$20 \times 60 \times \frac{2\pi}{2\pi} = 190.99 \text{ RPM}$$

Summary of Results:

Table- [1]

Wheel Diameter	Torque Required (Nm)	RPM Required
70 mm	0.175 Nm	272.84 RPM
100 mm	0.25 Nm	190.99 RPM

100 mm wheels were chosen to ensure better ground clearance, stability, and smooth mobility across various surfaces.

3.2.2 Robotic arm servo motor selection

Degrees of Freedom (DOF)

In robotics and mechanism design, the Degrees of Freedom (DOF) represent the number of independent parameters that define a system's configuration. For planar mechanisms, each rigid body (link) can move in 3 ways: two translational (X and Y directions) and one rotational.

To determine the DOF of a planar mechanism, we use Gruebler's Equation:

$$\text{DOF} = 3(N-1) - 2C$$

Where:

- N = Total number of links, including the base.
- C = Number of independent closed kinematic chains (loops).

Each closed loop introduces constraints that limit the system's mobility. As the number of closed chains increases, the overall freedom of the system decreases.

For a mechanism with:

- $N = 4$ links (including the base),
- $C = 2$ closed loops (independent constraints),

Apply the formula: $\text{DOF} = 3(4-1) - 2(2) = 9 - 4 = 5$

Thus, the system has 5 degrees of freedom, meaning it can independently perform 5 unique movements. This DOF value indicates high flexibility and controllability, ideal for robotic arms, parallel linkages, or grippers requiring precise multi-axis control.

Table- [2]

Joint	Type of Motion	Motor Used	Rotation
Base	Yaw (360° Rotation)	MG995	360°
Shoulder	Pitch (Up/Down)	MG995	180°
Elbow	Pitch (Up/Down)	MG995	180°
Wrist	Roll (360° Rotation)	SG90	360°
Gripper	Open/Close	SG90	180°

Torque calculation

Joint	Torque Required	Servo Used	Servo Torque
Wrist (SG90)	0.85 kg.cm	SG90	1.8 kg.cm
Elbow (MG995)	2.7 kg.cm	MG995	10 kg.cm
Shoulder (MG995)	4.8 kg.cm	MG995	10 kg.cm
Base (MG995)	6.6 kg.cm	MG995	10 kg.cm
Gripper (SG90)	N/A	SG90	1.8 kg.cm

Kinematics Analysis

A. Forward Kinematics (FK)

$$X = L1 \cdot \cos(\theta_1) + L2 \cdot \cos(\theta_1 + \theta_2) + L3 \cdot \cos(\theta_1 + \theta_2 + \theta_3)$$

$$Y = L1 \cdot \sin(\theta_1) + L2 \cdot \sin(\theta_1 + \theta_2) + L3 \cdot \sin(\theta_1 + \theta_2 + \theta_3)$$

B. Inverse Kinematics (IK)

$$\theta_1 = \text{atan2}(Y, X)$$

$$\theta_2 = \cos^{-1}((X^2 + Y^2 - L1^2 - L2^2) / (2 * L1 * L2))$$

$$\theta_3 = \text{desired wrist orientation angle} - (\theta_1 + \theta_2)$$

Force and Torque Calculator			
	Mass (lbs)	Length (in)	Center of Mass
Linkage 1	0.1212542	5.90551181	0.55115566
Linkage 2	0.1212542	1.96850394	0.04409245
Joint 2	1		
Object	1		
efficiency	65%		
Joint 1 Acc	360	Joint 2 Acc	180 deg/s^2
calculating:			
Joint 1 Torque	1000.696	lbs/in	torque for first servo
Joint 2 Torque	21.767	lbs/in	torque for second servo
servo	HS-255	has 26.878759 lbs/in	the servo I am using

Fig [1]- Force and Torque Calculator

Figure 1 depicts how forces are applied and how torque is calculated across the joints of the mechanism.

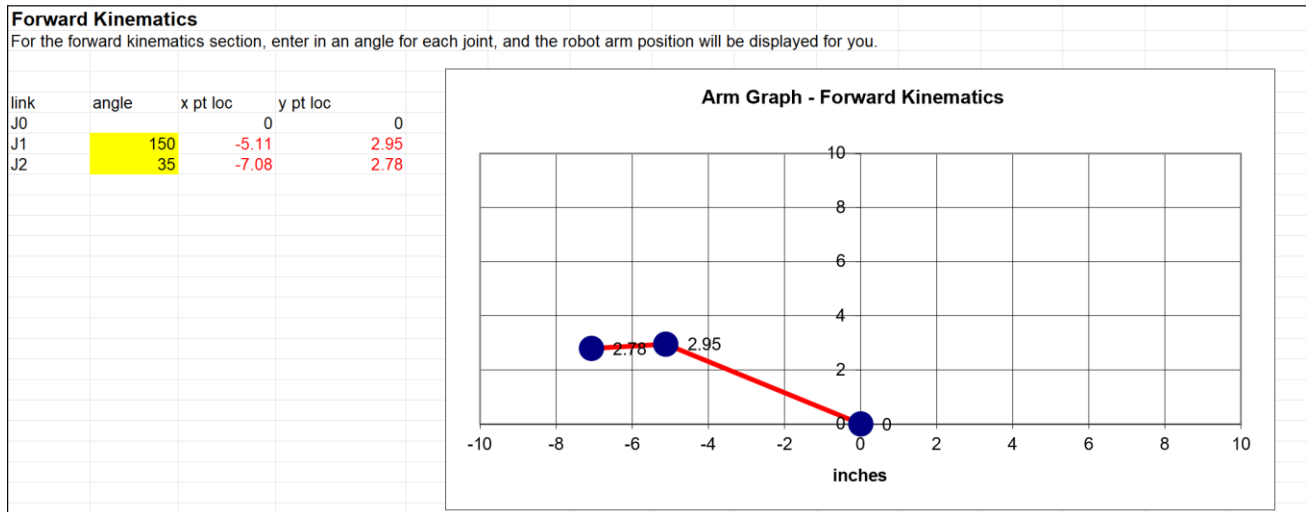


Fig [2] – Forward kinematics

Figure 2 illustrates the forward kinematics of the mechanism with joint angles J1 and J2 set to 150° and 35° , respectively.

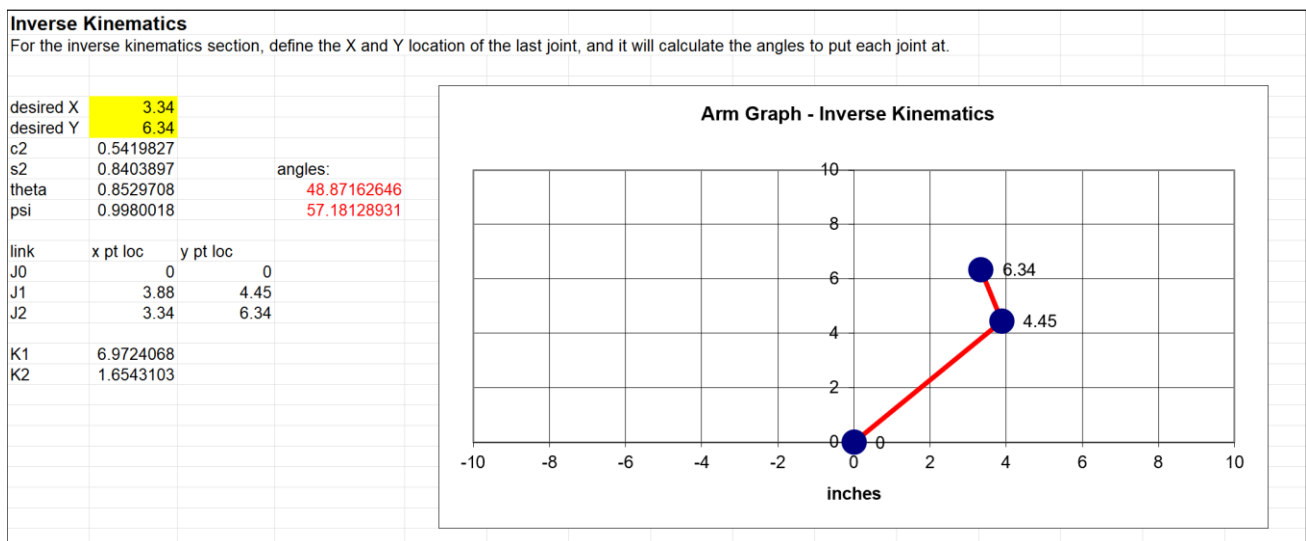


Fig [3]-Inverse kinematics

Figure 3 illustrates how the position of the end effector changes in response to variations in the X and Y coordinates.

3.2.3 Components descriptions

Raspberry pi 5 (4GB): The Raspberry Pi 5 is the robot's central processing unit with a quad-core 64-bit ARM Cortex-A76 processor at 2.4 GHz. It is equipped with up to 8 GB of LPDDR4X RAM, dual micro-HDMI ports with support for 4K displays, USB 3.0 ports with high-speed data transfer, a PCIe

interface, and a 40-pin GPIO header for interfacing with hardware. These features make it powerful enough to take care of vision processing, servo coordination, integration of sensors, and AI applications all at once, thus being a great platform for robotics and automation.

Figure [4]: Raspberry Pi 5



Camera Noir Module

The Camera Noir Module 3 is a 12MP camera with HDR capability, autofocus, and the NoIR (No Infrared Filter) feature, making it excel in low-light and IR-lit environments. Specifically designed for Raspberry Pi boards, this camera is suitable for computer vision applications such as face/object detection, tracking, and environmental awareness. The plug-and-play functionality and compact design with the Raspberry Pi make it an efficient and dependable tool for vision-based robotic applications.



Figure [5]: Raspberry Pi Camera Noir Module 3

AI Think Voice Recognition Module

The AI Think Voice Recognition Module is an independent, offline voice recognition module that provides customizable command input without the need for cloud connectivity. It recognizes voice commands locally, which makes it ideal for embedded systems such as robots. With built-in microphones and an easy UART or GPIO interface, this module enables the robot to recognize and

respond to specific voice commands, allowing hands-free operation and natural user interaction.



Figure [6]: AI Think Voice Recognition Module

Ultrasonic sensor

The ultrasonic sensor, in most cases an HC-SR04, provides distance measurement and object detection by sending ultrasonic waves and computing the time for the echo to come back. This sensor is essential for obstacle avoidance and safe navigation, enabling the robot to sense walls, objects, and users ahead. It provides accurate range detection between 2 cm and 400 cm and works well with microcontrollers and SBCs such as the Raspberry Pi.



Figure [7]: Ultrasonic Sensor

MG995

The MG995 is a high-torque servo motor with the ability to output up to 10 kg·cm of torque, ideal for lifting heavy loads and supporting critical joints such as the robot base and elbow. Powered by 4.8V to 6V, this servo delivers strong rotation and solid metal gear design for long-term reliability. Its durability under heavy mechanical load makes it a backbone element in robotic arm applications where strength and stable positioning are required.

SG90 Servo Motor

The SG90 is a low-torque, lightweight servo motor used for precision control in low-load environments. With a torque of about 1.8 kg·cm and a large operating angle, it is suited for uses such

as rotating the wrist or driving a gripper. Its low profile and energy efficiency make it ideal for use in smaller joints where agility and energy efficiency are more important than raw power.

L298N Motor Driver

The L298N is a dual H-bridge motor driver that can drive two DC motors separately. It has voltage support from 5V to 35V and can supply up to 2A per channel, which is ideal for medium-load applications such as mobile robots. The driver provides forward, reverse, and braking control through logic-level signals and is fully compatible with microcontrollers and single-board computers such as the Raspberry Pi.

16-bit Servo Driver (PCA9685)

The PCA9685 16-channel, 12-bit PWM servo driver allows more than the Raspberry Pi GPIO's limitations for controlling servos. Each of the channels is able to supply accurate control signals to individual servos, enabling synchronized movement in multi-joint robotic systems. The driver supports I2C communication, daisy-chaining, and very little pin usage for extensive servo control.

200 RPM Motors

These 200 revolutions per minute DC motors are used for balancing torque and speed in mobile robots. These are generally installed in wheeled robots to have smooth, reliable motion. Because they are easily compatible with ordinary motor drivers as well as gear attachments, integrating them is effortless and ground movement is reliable.

LiPo Battery

A Lithium Polymer (LiPo) battery offers high energy density, low weight, and stable output voltage, and it is thus well suited to power servo motors, sensors, and microcontrollers. The used battery is generally of 3S (11.1V or 12.6V when charged to capacity) configuration, delivering sufficient power for robotics applications with high loads while retaining portability and compactness.

3S BMS Module (12.6V, 25A)

3S Battery Management System safely charges and discharges 3-cell LiPo battery packs and guards against overcharging, over-discharging, overcurrent, and short circuit. This module, with its 25A rating, delivers stable and secure power to all robot components and prolongs battery life, while avoiding accidents.

18650 Batteries

18650 lithium-ion batteries are widely available for use in bespoke battery packs because of their large capacity, reusability, and convenient cylindrical shape. The cells give approximately 3.7V and 2000–3000mAh in most cases and are ideal to make high capacity power sources that are used by putting them into series and parallel circuits for applications involving robots.

JST / XT60 Connectors

JST and XT60 connectors provide secure, stable electrical connections among batteries, BMS modules, and power systems. JST connectors are for low-current signal applications and are small in size, whereas XT60 connectors can handle high current and are utilized in LiPo battery connections for stable and secure power delivery when under load.

12V-5A Power Adapter

12V-5A power adapter provides stable

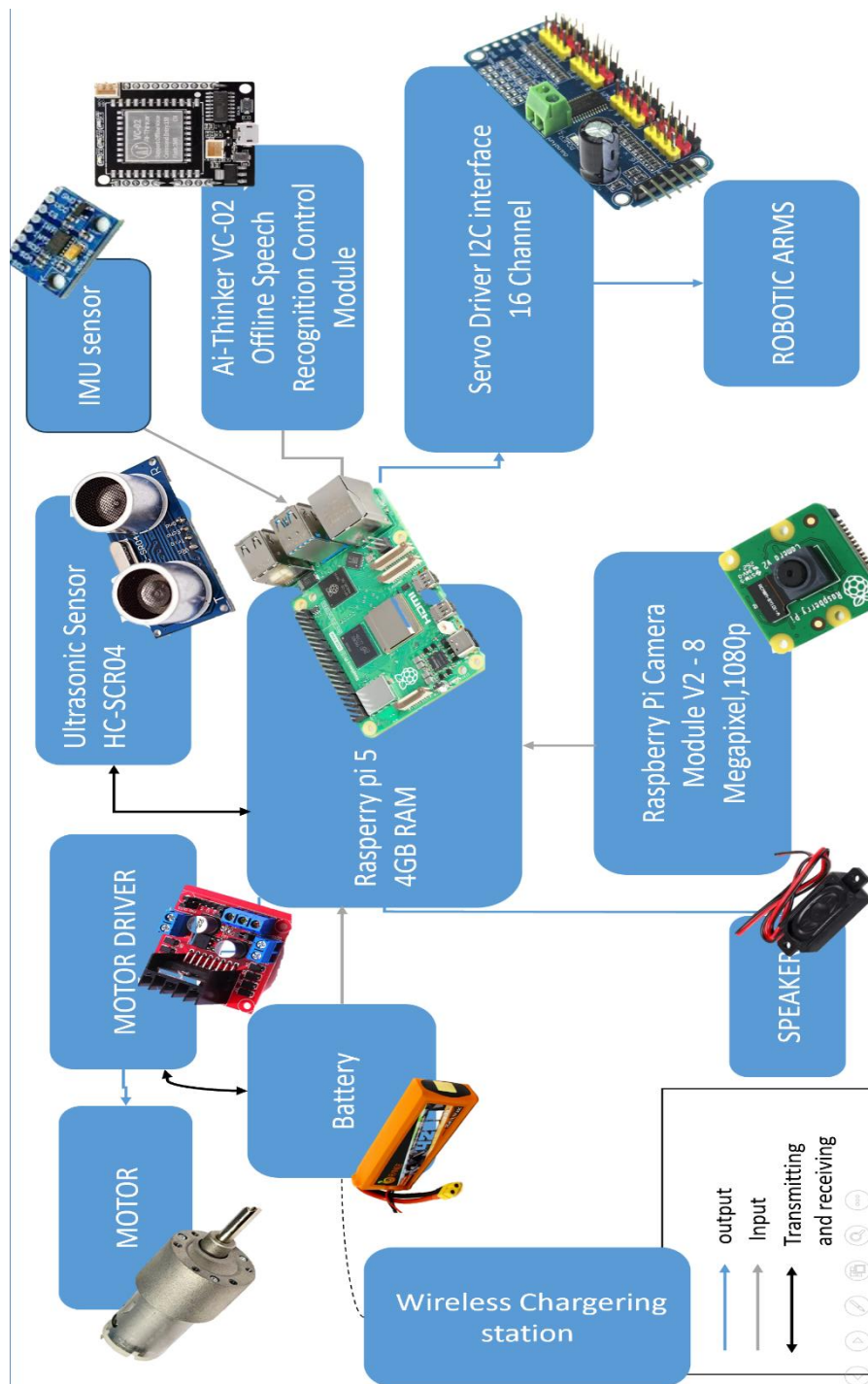
The parts for the robot were chosen through precise calculations of torque, degrees of freedom, and power needs to guarantee the best performance. High-torque MG995 servos were used in the base and elbow joints to manage the payload and arm load, while SG90 servos were used in the wrist and gripper owing to their lower torque requirements. A 5 DOF configuration was deemed enough through mobility calculations, providing manipulation flexibility. Power system parts, such as a 12V-5A adapter, XL4015 step-down converter, and a 3S BMS for LiPo batteries, were chosen based on estimated current and voltage demands. The Raspberry Pi 5 was selected as the main controller to handle intricate processing, in addition to a NoIR Camera Module 3 and AI Think Voice Module for vision and voice control, making sure that every component is compatible with the robot's mechanical and functional calculations.

Table [3]

S.No	Component		Description / Specification	Required
1	MG995 Servo Motor		Torque: 10 kg.cm, used for base and elbow joints	4
2	SG90 Servo Motor		Torque: 1.8 kg.cm, used for wrist joint and gripper	8
3	Laser-Cut Sheets	Acrylic	Structural material for the robot's frame and joints	1
4	Raspberry Pi 5		Central controller	1
5	Raspberry Pi Camera Noir Module 3		For vision-based applications	1
6	AI Think Voice Recognition Module		Enables voice command functionality	1
7	Ultrasonic Sensor		Assists in distance measurement	1
8	L298N Motor Driver		Controls motor movement	
9	16-bit Servo Driver		Manages multiple servo motors	1
10	200 RPM Motors		Provides robot mobility and speed	1
11	LiPo Battery		Powers the robot system	1
12	3S BMS Module (12.6V, 25A)		Manages charging and discharging of 18650 batteries	1
13	18650 Batteries		Rechargeable cells used in battery pack	1
14	JST / Connectors	XT60	Secure wiring for battery connections	1
15	12V-5A Adapter	Power	Power supply for the charging station	1
16	XL4015 DC-DC Step-down Converter		With voltmeter; regulates voltage	1
17	Copper Strips / Tape		Used for contact-based charging	1
18	Schottky Diode (1N5822, 10A)		Prevents reverse current	1
19	10A Fuse		Provides overcurrent protection	1

20	12V Indicator Resistor	Charging LED +	Visual indication of charging status	1
21	DC Barrel Connector	Jack	Connects adapter to charging circuit	1

BLOCK DIAGRAM



3.3 CAD MODELING

The robot is designed to accommodate all essential components efficiently while ensuring proper ventilation for optimal performance and heat management shown in fig 8&9.

Fig [8] : Robot CAD model front view



Fig [9] : Robot CAD model side view



This DXF file is prepared specifically for laser cutting the acrylic sheet components required for assembling the robot's structure.

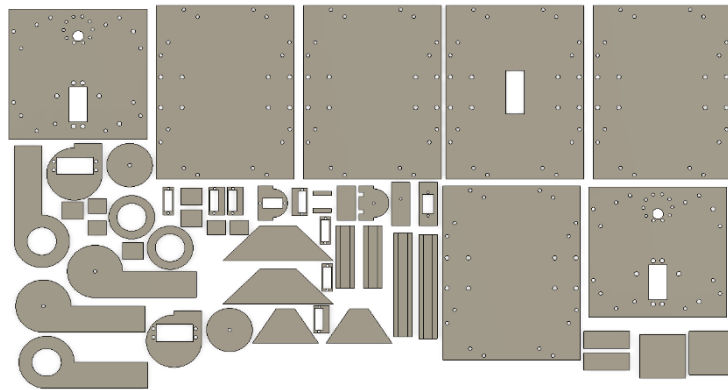


Fig [10] : DXF file of robot

The charging station STL file will include a base platform with mounting points for charging contacts, a central alignment slot for the robot, and space for copper strips or tape to ensure efficient contact-based charging shown in fig 10.

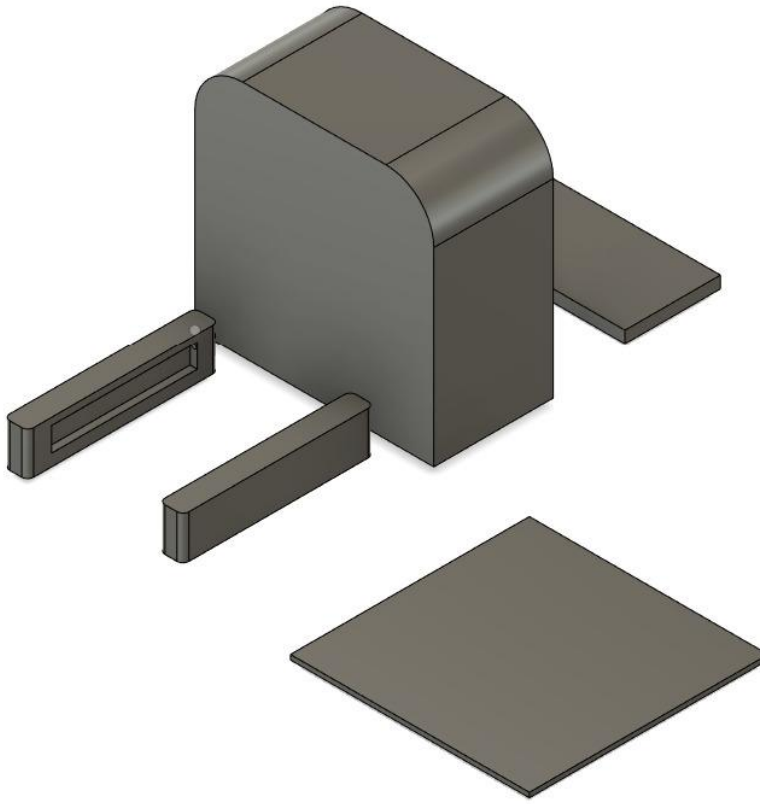


Fig [11] Charging station STLE file

Fig 11 depicts the charging station STLE file.

3.4 ALGORITHM SELECTION

3.4.1 OBJECT AND FACE DETECTION

Object detection models typically fall into two categories: single-stage and two-stage detectors. Single-stage models like YOLO (You Only Look Once), SSD (Single Shot MultiBox Detector), and EfficientDet detect objects in a single pass, providing fast speed and low latency, and hence are very well-suited for real-time applications such as robotics, autonomous drones, and CCTV surveillance. Conversely, two-stage models such as Faster R-CNN, Mask R-CNN, and R-FCN generate region proposals and then classify them, which is more accurate but slower. These are best suited for precision-demanding applications such as medical imaging, satellite imagery, and autonomous driving.

Of single-stage detectors, YOLO is now incredibly popular because of its remarkable speed-accuracy balance. It has progressed from YOLOv1 to YOLOv8, with every iteration bringing novel architectural advancements, improved backbone networks, and the ability to handle advanced tasks. YOLOv8, from Ultralytics, is especially versatile—it not only handles object detection but also image classification, segmentation, and pose estimation. In YOLOv8, YOLOv8s (small) provides the optimal balance between real-time capabilities and detection performance and is therefore well-suited for use in embedded systems, low-power robots, and mobile AI platforms.

Besides general object detection, face detection is a specific task that has improved significantly with the use of one-stage and two-stage models. One-stage models such as YOLOv5 and YOLOv8 can be fine-tuned for real-time face detection and deliver good results in applications including security systems, attendance monitoring, and human-robot interaction. For more sophisticated uses involving facial landmark detection or emotion analysis, one can employ dedicated models such as MTCNN, RetinaFace, or BlazeFace, typically as extensions or complementing networks to detection models.

Overall, for real-time applications such as personal assistive robots, mobile surveillance vehicles, or interactive information kiosks, YOLOv8s offers a strong solution to object and face detection, both speed and accuracy balanced, while offering flexibility with a variety of vision tasks. For accuracy-dependent applications, merging or converting to two-stage models can offer additional accuracy needed, particularly when used in conjunction with face-specific detection models.



YOLOv8 is selected because it has a lightweight architecture and efficient inference time, making it perfect for embedding and running on resource-limited environments. Its small version, YOLOv8s, achieves a good balance between performance and accuracy and is well suited to run in real-time efficient object and face detection. Due to its support for multiple tasks such as classification, segmentation, and pose estimation, YOLOv8 is very versatile. The model's optimized architecture facilitates easy deployment on hardware such as Raspberry Pi, NVIDIA Jetson, or mobile chips. Its velocity provides effortless incorporation into robotics where response times matter. In addition, YOLOv8's open-source environment and constant updates by Ultralytics render it developer-friendly and production-ready.

3.4.2 A* ALGORITHM

This project uses the A* (A-star) algorithm for optimal path planning and navigation. A* is an informed search algorithm that has the advantages of both Dijkstra's algorithm and the Greedy Best-First Search. A* estimates paths by using a cost function:

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the real cost from the starting node to the present node, and $h(n)$ is an estimation of the heuristic cost from the present node to the destination. Balancing both cost and heuristic prediction, A* guarantees an optimal and efficient route without collisions, and it's perfect for real-time robot navigation.

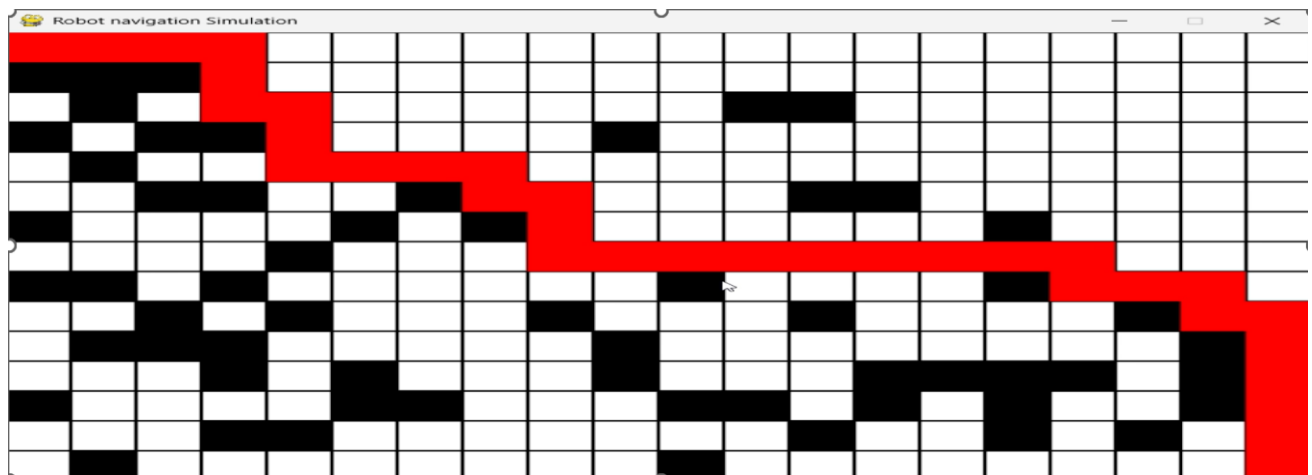


FIG [12]- A* ALGORITHM IN NUMP PI

Fig 12 shows the path planning in python using pygame.

CHAPTER 4

PROGRAMMING

4.1 IMAGE CAPTURING

Libraries used

- cv2: Used for image display and saving using OpenCV.
- os: Facilitates folder and path handling.
- datetime: Used for timestamping each photo file.
- Picamera2: Allows access to the Raspberry Pi Camera Module 3.
- time: Provides functions to pause or wait during camera startup.

PERSON_NAME = "sameer" This variable sets the name of the person being photographed, which is later used to create a dedicated folder for saving that person's images.

```
def create_folder(name):  
    dataset_folder = "dataset"  
    if not os.path.exists(dataset_folder):  
        os.makedirs(dataset_folder)  
  
    person_folder = os.path.join(dataset_folder, name)  
    if not os.path.exists(person_folder):  
        os.makedirs(person_folder)  
    return person_folder
```

Explanation:

- The function create_folder(name) creates a directory structure to store images.
- It first checks if a folder named dataset exists; if not, it creates one.
- Then, it creates (or verifies the existence of) a subfolder within dataset that is named after the person.
- Finally, it returns the path to the person's folder.

```
def capture_photos(name):  
    folder = create_folder(name)
```

Explanation:

This line calls the create_folder function to create and retrieve a folder where the captured photos will be saved.

```

picam2 = Picamera2()
picam2.configure(picam2.create_preview_configuration(main={"format": 'XRGB8888',
"size": (640, 480)}))
picam2.start()

```

Explanation:

- The Pi Camera (Picamera2) is initialized.
- The camera is configured to capture images in the 'XRGB8888' format at a resolution of 640x480.
- The camera is then started to begin capturing video frames.

```
time.sleep(2)
```

Explanation:

This pauses the program for 2 seconds, allowing the camera to warm up before capturing images.

```

photo_count = 0
print(f"Taking photos for {name}. Press SPACE to capture, 'q' to quit.")

```

Explanation:

- The variable photo_count is initialized to zero to keep track of the number of photos taken.
- A message is printed to inform the user how to capture images (press SPACE) and how to quit (press 'q').

```

while True:
    frame = picam2.capture_array()

```

Explanation:

- This begins an infinite loop where video frames are continuously captured from the camera using picam2.capture_array().

```
cv2.imshow('Capture', frame)
```

Explanation:

- The current frame is displayed in a window titled "Capture" using OpenCV.

```
key = cv2.waitKey(1) & 0xFF
```

Explanation:

- The program waits for 1 millisecond for any key press and captures the pressed key (if any).

```

if key == ord(' '): # Space key
    photo_count += 1
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    filename = f"{name}_{timestamp}.jpg"

```

```

filepath = os.path.join(folder, filename)
cv2.imwrite(filepath, frame)
print(f"Photo {photo_count} saved: {filepath}")

```

Explanation:

- If the Space key is pressed:
 - The photo counter is incremented.
 - A timestamp is generated using the current date and time.
 - A filename is created combining the person's name and the timestamp.
 - The full path for the image is created by joining the folder path with the filename.
 - The current frame is saved as a JPEG file at the specified path.
 - A message confirming the photo is printed.

```

elif key == ord('q'): # Q key
    break

```

Explanation:

- If the 'q' key is pressed, the loop is broken, and the script proceeds to exit.

```

cv2.destroyAllWindows()
picam2.stop()
print(f"Photo capture completed. {photo_count} photos saved for {name}.")

```

Explanation:

- After exiting the loop, all OpenCV windows are closed.
- The camera is stopped to free up resources.
- A message is printed indicating that photo capture is complete and shows the total number of photos saved.

```

import cv2
import os
from datetime import datetime
from picamera2 import Picamera2
import time

# Change this to the name of the person you're photographing
PERSON_NAME = "sameer"

def create_folder(name):
    dataset_folder = "dataset"

```

```

if not os.path.exists(dataset_folder):
    os.makedirs(dataset_folder)

person_folder = os.path.join(dataset_folder, name)
if not os.path.exists(person_folder):
    os.makedirs(person_folder)
return person_folder

def capture_photos(name):
    folder = create_folder(name)

    # Initialize the camera
    picam2 = Picamera2()
    picam2.configure(picam2.create_preview_configuration(main={"format": 'XRGB8888', "size":
(640, 480)}))
    picam2.start()

    # Allow camera to warm up
    time.sleep(2)

    photo_count = 0

    print(f"Taking photos for {name}. Press SPACE to capture, 'q' to quit.")

    while True:
        # Capture frame from Pi Camera
        frame = picam2.capture_array()

        # Display the frame
        cv2.imshow('Capture', frame)

        key = cv2.waitKey(1) & 0xFF

        if key == ord(' '): # Space key

```

```

    photo_count += 1
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    filename = f'{name}_{timestamp}.jpg'
    filepath = os.path.join(folder, filename)
    cv2.imwrite(filepath, frame)
    print(f"Photo {photo_count} saved: {filepath}")

elif key == ord('q'): # Q key
    break

# Clean up
cv2.destroyAllWindows()
picam2.stop()
print(f"Photo capture completed. {photo_count} photos saved for {name}.")

if __name__ == "__main__":
    capture_photos(PERSON_NAME)

```

Below is a detailed line-by-line explanation of your Raspberry Pi face dataset collection script using the Pi Camera

```

import cv2
import os
from datetime import datetime
from picamera2 import Picamera2
import time

```

4.2 MODEL TRAINING

- `os`: Helps in file and folder path manipulation.
- `imutils.paths`: Used to list all image file paths from a directory.
- `face_recognition`: A Python library for face detection and recognition.
- `pickle`: Used for saving (serializing) Python objects to a file.

- cv2: OpenCV for image reading and conversion.

Start Message

```
print("[INFO] start processing faces...")
```

- Prints a message indicating that the face processing is starting.

Load All Image Paths

```
imagePaths = list(paths.list_images("dataset"))
```

```
knownEncodings = []
```

```
knownNames = []
```

- `paths.list_images("dataset")` generates a list of all image file paths found in the "dataset" folder.
- `knownEncodings`: An empty list that will store the face encodings.
- `knownNames`: An empty list that will store the corresponding names for each encoding.

Process Each Image

```
for (i, imagePath) in enumerate(imagePaths):
```

```
    print(f"[INFO] processing image {i + 1}/{len(imagePaths)}")
```

- Iterates over each image path using a loop.
- Prints progress information, indicating which image out of the total is currently being processed.

Extract Person's Name from Path

```
name = imagePath.split(os.path.sep)[-2]
```

- Splits the image path by the directory separator and extracts the second-to-last element as the person's name (assuming that the folder name represents the person's name).

Read and Convert Image

```
image = cv2.imread(imagePath)
```

```
rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

- Reads the image file using OpenCV.
- Converts the image from BGR color space (OpenCV's default) to RGB color space required by the face_recognition library.

Detect Faces and Compute Encodings

```
boxes = face_recognition.face_locations(rgb, model="hog")
```

```
encodings = face_recognition.face_encodings(rgb, boxes)
```

- Detects face locations in the image using the HOG model, which is efficient for CPU-based systems.
- Computes a 128-dimension encoding for each detected face, representing its unique features.

Store Encodings

```
for encoding in encodings:
```

```
    knownEncodings.append(encoding)
```

```
    knownNames.append(name)
```

- For each detected face encoding, the encoding is appended to the knownEncodings list.
- The corresponding name (extracted from the folder path) is appended to the knownNames list.

Save Encodings to File

```
print("[INFO] serializing encodings...")
```

```
data = {"encodings": knownEncodings, "names": knownNames}
```

```
with open("encodings.pickle", "wb") as f:
```

```
    f.write(pickle.dumps(data))
```

- Prints a message indicating that the serializing (saving) process is starting.
- Creates a dictionary containing the face encodings and their names.
- Serializes the dictionary using pickle and writes it to a file named "encodings.pickle".

Done Message

```
print("[INFO] Training complete. Encodings saved to 'encodings.pickle'")
```

- Prints a confirmation message stating that the face encoding process is complete and that the encodings have been saved.

Result: After running the script, you will have a file named "encodings.pickle" that contains the facial data (encodings and corresponding names) for your dataset. This file can later be loaded for performing face recognition in real time.

```
import os
```

```
from imutils import paths
```

```
import face_recognition
```

```
import pickle
```

```
import cv2
```

```
print("[INFO] start processing faces...")
```

```
imagePaths = list(paths.list_images("dataset"))
```

```
knownEncodings = []
```

```
knownNames = []
```

```
for (i, imagePath) in enumerate(imagePaths):
```

```
    print(f"[INFO] processing image {i + 1}/{len(imagePaths)}")
```

```
    name = imagePath.split(os.path.sep)[-2]
```

```
    image = cv2.imread(imagePath)
```

```
    rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

```
    boxes = face_recognition.face_locations(rgb, model="hog")
```

```
    encodings = face_recognition.face_encodings(rgb, boxes)
```

```
    for encoding in encodings:
```

```
        knownEncodings.append(encoding)
```



```
knownNames.append(name)
```

```
print("[INFO] serializing encodings...")
```

```
data = {"encodings": knownEncodings, "names": knownNames}
```

```
with open("encodings.pickle", "wb") as f:
```

```
    f.write(pickle.dumps(data))
```

```
print("[INFO] Training complete. Encodings saved to 'encodings.pickle'")
```

Below is a detailed line-by-line breakdown of your face encoding script

Library Imports

```
import os
```

```
from imutils import paths
```

```
import face_recognition
```

```
import pickle
```

```
import cv2
```

4.3 FACE RECOGNITION

Importing Required Libraries

```
import face_recognition
```

```
import cv2
```

```
import numpy as np
```

```
from picamera2 import Picamera2
```

```
import time
```

```
import pickle
```

- These libraries serve the following purposes:
 - **face_recognition**: Provides functions for detecting and recognizing faces.
 - **cv2 (OpenCV)**: Used for image capture, display, and drawing annotations.

- **numpy**: Supports efficient array operations necessary for image processing.
- **Picamera2**: Controls the Raspberry Pi Camera (Noir Module 3).
- **time**: Provides time functions, such as pausing execution or measuring elapsed time.
- **pickle**: Handles serialization and deserialization of Python objects (used for loading saved face encodings).

Load Known Encodings

```
print("[INFO] loading encodings...")

with open("encodings.pickle", "rb") as f:

    data = pickle.loads(f.read())

known_face_encodings = data["encodings"]

known_face_names = data["names"]
```

- A message is printed to indicate that the script is loading face encodings.
- The file "encodings.pickle" is opened in binary read mode, and its contents are loaded using pickle.
- The variable known_face_encodings is assigned the list of facial feature vectors, and known_face_names is assigned the corresponding names. These are the “trained” data for face recognition.

Initialize Pi Camera

```
picam2 = Picamera2()

picam2.configure(picam2.create_preview_configuration(main={"format": 'XRGB8888', "size":
(1920, 1080)}))

picam2.start()
```

- A new instance of the Pi Camera is created.
- The camera is configured for preview mode with a resolution of 1920x1080 and an image format of XRGB8888. This high resolution improves detection accuracy.
- The camera is started to begin capturing frames.

Initialize Tracking Variables

```
cv_scaler = 4 # Scale factor to downsize frame for faster processing
```

```
face_locations = []
```

```
face_encodings = []
```

```
face_names = []
```

```
frame_count = 0
```

```
start_time = time.time()
```

```
fps = 0
```

- cv_scaler is set to 4, which downsizes each frame to increase processing speed by reducing the number of pixels processed.
- Lists for face_locations, face_encodings, and face_names are initialized to store data for each frame.
- Variables frame_count, start_time, and fps are initialized for measuring the frames per second (FPS).

Function to Process Each Frame

```
def process_frame(frame):
```

This function is responsible for:

1. Downsizing the frame to improve processing speed.
2. Converting the frame from BGR (used by OpenCV) to RGB (required by the face_recognition library).
3. Detecting faces and computing their encodings.
4. Comparing detected encodings with known face encodings.
5. Identifying each face by name or marking it as "Unknown."

Function to Draw Results on the Frame

```
def draw_results(frame)
```

- This function draws the detected results on the frame by:
 - Drawing bounding boxes around each detected face.
 - Displaying the corresponding name for each recognized face.
 - Scaling the face location coordinates back up to the original frame size using the `cv_scaler`.

Function to Calculate Frame Per Second (FPS)

`def calculate_fps():`

- This function tracks the number of frames processed over a period of time and calculates the frames per second (FPS) for monitoring the performance of the face recognition system.

Main Loop: Real-Time Detection

`while True:`

`frame = picam2.capture_array()`

`processed_frame = process_frame(frame)`

`display_frame = draw_results(processed_frame)`

`current_fps = calculate_fps()`

- The script enters an infinite loop, where:
 - A frame is continuously captured from the Pi Camera.
 - The frame is processed to detect and recognize faces.
 - The processed results (bounding boxes, names, etc.) are drawn on the frame.
 - The FPS is calculated based on the number of frames processed per second.

Display Frame + Quit on 'Q'

`cv2.putText(display_frame, f"FPS: {current_fps:.1f}", ...)`

`cv2.imshow('Video', display_frame)`

`if cv2.waitKey(1) == ord("q"):`

break

- The current FPS is overlaid on the display frame.
- The annotated frame is shown in a window titled "Video."
- The loop continuously checks for a key press, and if the 'q' key is pressed, the loop terminates.

Clean-Up

cv2.destroyAllWindows()

picam2.stop()

- After the loop exits, all OpenCV windows are closed.
- The Pi Camera is stopped to release resources properly.

```
import face_recognition
```

```
import cv2
```

```
import numpy as np
```

```
from picamera2 import Picamera2
```

```
import time
```

```
import pickle
```

```
Load pre-trained face encodings
```

```
print("[INFO] loading encodings...")
```

```
with open("encodings.pickle", "rb") as f:
```

```
data = pickle.loads(f.read())
```

```
known_face_encodings = data["encodings"]
```

```
known_face_names = data["names"]
```

```
Initialize the camera
```

```
picam2 = Picamera2()
```

```
picam2.configure(picam2.create_preview_configuration(main={"format": 'XRGB8888', "size":  
(1920, 1080)}))
```

```
picam2.start()
```

```
Initialize our variables
```

```
cv_scaler = 4 # this has to be a whole number
```

```

face_locations = []
face_encodings = []
face_names = []
frame_count = 0
start_time = time.time()
fps = 0
def process_frame(frame):
    global face_locations, face_encodings, face_names
    # Resize the frame using cv_scaler to increase performance (less pixels processed, less time
    spent)
    resized_frame = cv2.resize(frame, (0, 0), fx=(1/cv_scaler), fy=(1/cv_scaler))
    # Convert the image from BGR to RGB colour space, the facial recognition library uses RGB,
    OpenCV uses BGR
    rgb_resized_frame = cv2.cvtColor(resized_frame, cv2.COLOR_BGR2RGB)
    # Find all the faces and face encodings in the current frame of video
    face_locations = face_recognition.face_locations(rgb_resized_frame)

    face_encodings = face_recognition.face_encodings(rgb_resized_frame, face_locations,
    model='large')
    face_names = []
    for face_encoding in face_encodings:
        # See if the face is a match for the known face(s)
        matches = face_recognition.compare_faces(known_face_encodings, face_encoding)
        name = "Unknown"
        # Use the known face with the smallest distance to the new face
        face_distances = face_recognition.face_distance(known_face_encodings, face_encoding)
        best_match_index = np.argmin(face_distances)
        if matches[best_match_index]:
            name = known_face_names[best_match_index]
        face_names.append(name)
    return frame

```

```

def draw_results(frame):
# Display the results
for (top, right, bottom, left), name in zip(face_locations, face_names):
    # Scale back up face locations since the frame we detected in was scaled
    top *= cv_scaler
    right *= cv_scaler
    bottom *= cv_scaler
    left *= cv_scaler
    # Draw a box around the face
    cv2.rectangle(frame, (left, top), (right, bottom), (244, 42, 3), 3)
    # Draw a label with a name below the face
    cv2.rectangle(frame, (left - 3, top - 35), (right+3, top), (244, 42, 3), cv2.FILLED)
    font = cv2.FONT_HERSHEY_DUPLEX
    cv2.putText(frame, name, (left + 6, top - 6), font, 1.0, (255, 255, 255), 1)
return frame

def calculate_fps():
global frame_count, start_time, fps
frame_count += 1
elapsed_time = time.time() - start_time
if elapsed_time > 1:
    fps = frame_count / elapsed_time
    frame_count = 0
    start_time = time.time()
return fps

while True:
# Capture a frame from camera
frame = picam2.capture_array()
# Process the frame with the function
processed_frame = process_frame(frame)
# Get the text and boxes to be drawn based on the processed frame

```

```

display_frame = draw_results(processed_frame)
# Calculate and update FPS
current_fps = calculate_fps()
# Attach FPS counter to the text and boxes
cv2.putText(display_frame, f"FPS: {current_fps:.1f}", (display_frame.shape[1] - 150, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

# Display everything over the video feed.
cv2.imshow('Video', display_frame)
# Break the loop and stop the script if 'q' is pressed
if cv2.waitKey(1) == ord("q"):
    break

By breaking the loop we run this code here which closes everything
cv2.destroyAllWindows()
picam2.stop()

```

Summary

This script utilizes pre-trained face encodings to recognize faces in real-time. It captures live video from the Raspberry Pi Camera, processes each frame to detect and identify faces, and displays the results along with an FPS counter. The script is structured to provide a real-time face recognition system that continuously updates until manually stopped by pressing the 'q' key.

Let me know if further modifications or explanations are needed.

4.4 OBJECT DETECTION

Below is a detailed, plain-text explanation of the provided code:

1. Import Libraries
2. from ultralytics import YOLO
3. import cv2
4. from picamera2 import Picamera2

5. `import time`

- The code imports the YOLO model interface from the ultralytics package.
- `cv2` (OpenCV) is imported for image display and processing.
- `Picamera2` is used to access and control the Raspberry Pi Camera.
- `time` provides time functions such as `sleep`.

6. Load the YOLO Model

7. `model = YOLO('yolov8n.pt')`

- This line loads a pre-trained YOLOv8 "nano" model from the file 'yolov8n.pt'.
- The YOLOv8n model is chosen for its speed and efficiency, suitable for real-time inference on resource-constrained devices.

8. Initialize the Pi Camera

9. `picam2 = Picamera2()`

10. `picam2.preview_configuration.main.size = (640, 480)`

11. `picam2.preview_configuration.main.format = "RGB888"`

12. `picam2.configure("preview")`

13. `picam2.start()`

- An instance of the Pi Camera is created.
- The camera's preview configuration is set to a resolution of 640x480 pixels.
- The image format is configured as "RGB888" (which means it outputs 8-bit each for red, green, and blue channels).
- The camera is configured for preview mode and then started, which begins capturing frames.

14. Pause for Camera Warm-Up

15. `time.sleep(1)`

- The program pauses for 1 second to allow the camera sensor to stabilize before processing frames.

16. Main Loop for Live Object Detection

17. while True:

18. `frame = picam2.capture_array()`

19. `# Run YOLOv8 detection`

20. `results = model(frame, verbose=False)[0]`

21. `# Draw results on frame`

22. `annotated_frame = results.plot()`

23. `cv2.imshow("YOLOv8 Live Detection", annotated_frame)`

24. `if cv2.waitKey(1) & 0xFF == ord('q'):`

25. `break`

- The loop continuously captures frames from the camera using `picam2.capture_array()`.
- For each captured frame, the YOLO model is used to perform object detection. The call to `model(frame, verbose=False)[0]` returns the detection results for the frame.
- The `results.plot()` method draws the detection results (such as bounding boxes and labels) onto the frame.
- The annotated frame is then displayed in a window titled "YOLOv8 Live Detection".
- The loop checks every frame to see if the 'q' key is pressed. If detected, the loop is exited.

26. Clean Up Resources

27. `cv2.destroyAllWindows()`

- After the loop ends (when 'q' is pressed), all OpenCV windows are closed.
- Finally, the camera is stopped (code not shown in the snippet after the loop, but it is implied as a cleanup step).

```

from ultralytics import YOLO
import cv2
from picamera2 import Picamera2
import time

Load model (use YOLOv8n or YOLOv8s for fast inference)

model = YOLO('yolov8n.pt')

Initialize Pi Camera

picam2 = Picamera2()
picam2.preview_configuration.main.size = (640, 480)
picam2.preview_configuration.main.format = "RGB888"
picam2.configure("preview")
picam2.start()

time.sleep(1)

while True:
    frame = picam2.capture_array()

    # Run YOLOv8 detection

    results = model(frame, verbose=False)[0]

    # Draw results on frame

    annotated_frame = results.plot()

    cv2.imshow("YOLOv8 Live Detection", annotated_frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

    cv2.destroyAllWindows()

```

This script captures a live video stream from the Raspberry Pi Camera, applies YOLOv8 for object detection on each frame, and displays the annotated live feed until the user stops the program by pressing 'q'.

CHAPTER 5

RESULT

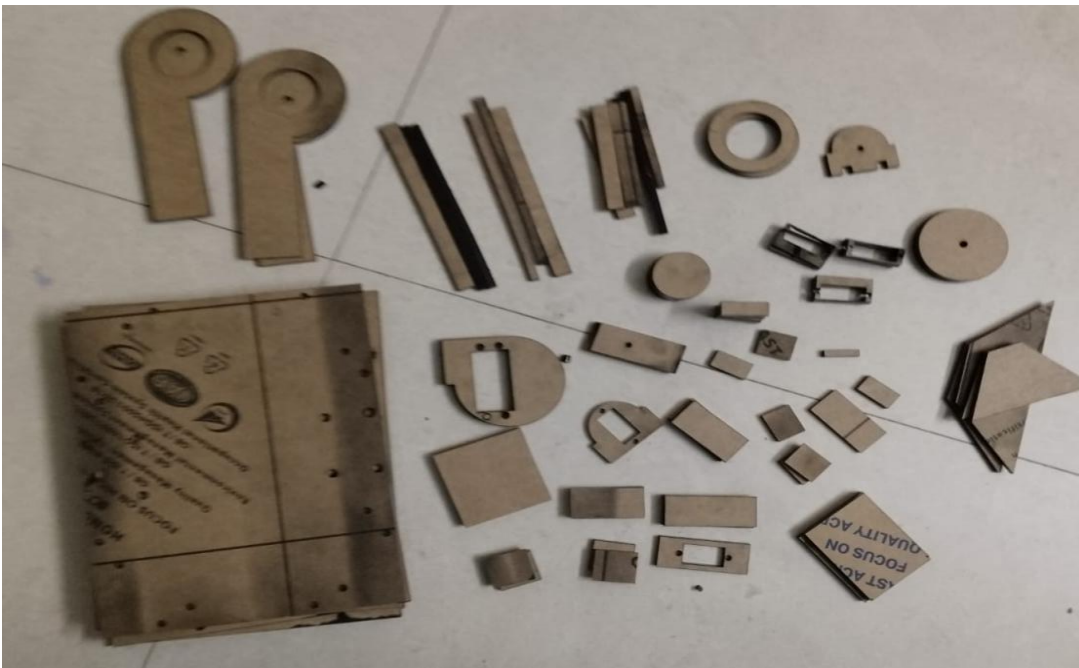
5.1 HARDWARE ASSEMBLY



FIG [13]-GIPPER 3D PRINTED



FIG [14]-CHARGING STATION 3D PRINTED



FIG[15] ARYCLIC SHEETS

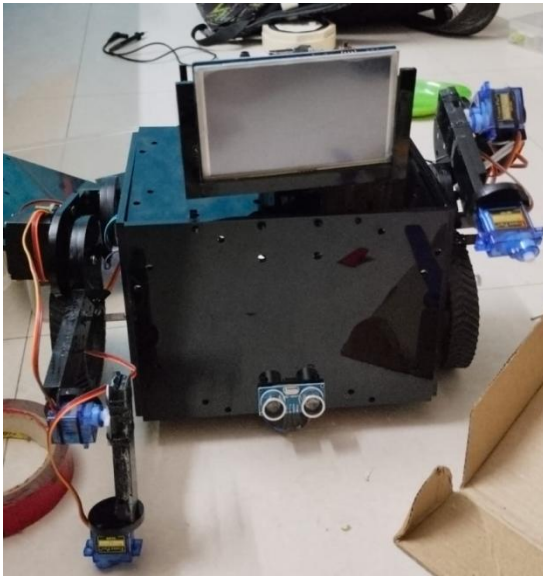


FIG [16]-FULLY ASSEMBLED

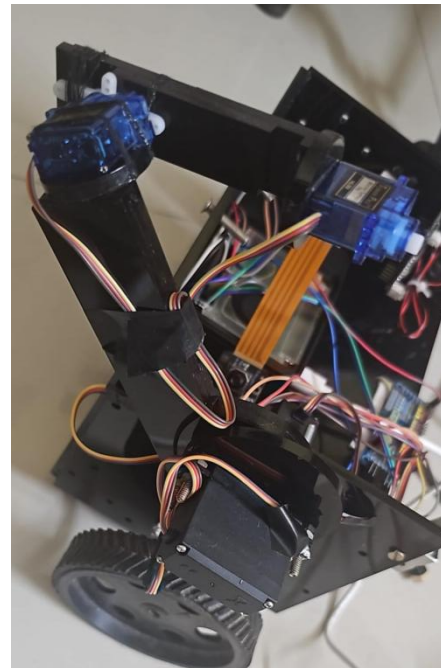


FIG [17]-ROBOTIC ARM

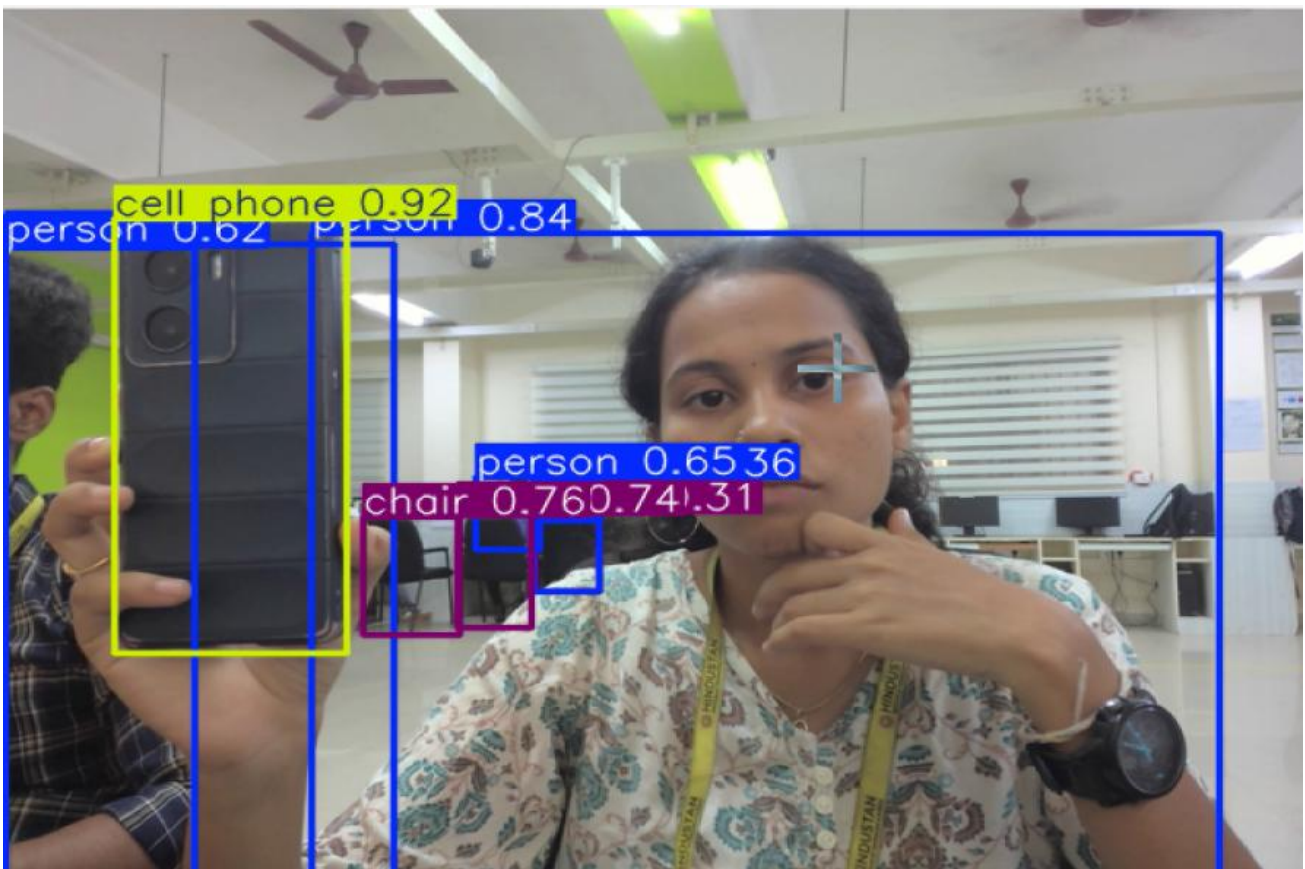


FIG [18]OBJECT DETECTION

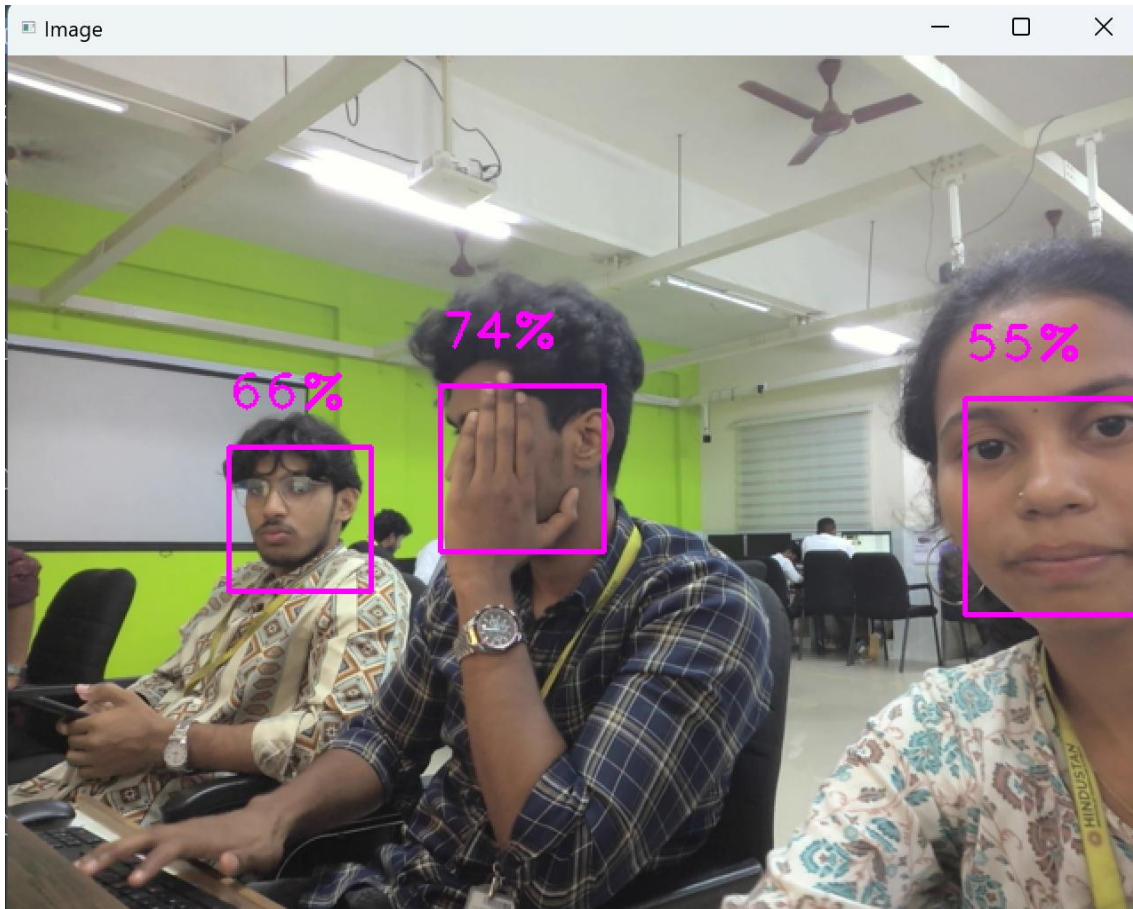


FIG [19] FACE DETECTION

5.1 Hardware Assembly

Figure 13 and Figure 14 illustrate the various 3D printed components used in the development of the robotic system. Figure 13 shows the individual 3D printed parts for the gripper mechanism, which include interlocking gears, brackets, and linkages required for gripping and manipulating objects. Figure 14 displays the 3D printed components for the charging station, including structural panels and support elements designed to house the electronics and facilitate easy docking of the robot for recharging purposes. These precisely printed parts were essential for assembling a functional and structurally sound robotic system.

The third image under this section displays several laser-cut MDF (Medium Density Fiberboard) parts that were used for constructing various mechanical structures of the robot. These parts include frame

components, wheel holders, arm segments, and other structural elements essential for the robot's overall chassis and movement system. The combination of MDF and 3D printed parts enabled a hybrid build, balancing strength and design flexibility.

5.2 Fully Assembled System and Features

Figure 16 shows the fully assembled robot with all major components integrated, including the body frame, robotic arms, wheels, and sensors. A screen is mounted at the front of the robot, likely used for interfacing or displaying information. The presence of ultrasonic sensors on the front indicates obstacle detection capability, while the dual robotic arms suggest manipulative abilities.

Figure 17 focuses specifically on the robotic arm, highlighting the multiple servo motors used for actuation and the wiring necessary for motor control and feedback. This mechanical setup enables the robot to perform complex tasks like gripping, lifting, and placing objects, making it suitable for assistive and interactive roles.

5.3 Object and Face Detection Capabilities

Figure 18 demonstrates the robot's object detection capabilities, powered by a deep learning model (most likely YOLOv11n or a similar real-time object detection algorithm). The system successfully identifies multiple objects in the scene including people, a cellphone, and a chair, with associated confidence scores. This feature is critical for enabling the robot to understand its surroundings and interact meaningfully with the environment.

Figure 19 showcases the face detection module in action. It detects and outlines multiple human faces in a classroom-like setting, providing confidence percentages for each face. This capability is essential for applications such as user recognition, interaction personalization, and human-following tasks, contributing to a more interactive and intelligent robot.

CHAPTER 6

CONCLUSION

The creation of the Personal Assistive Robot marks an important step towards incorporating intelligent systems into our daily lives for added convenience, interaction, and functionality. With the integration of computer vision, voice recognition, robotic manipulation, and autonomous mobility, the robot shows a multidisciplinary solution to real-world assistive technology problems.

The project successfully deployed two strong path planning algorithms—A* and RRT—both tested for their appropriateness in navigation in constrained indoor spaces. YOLOv8 was utilized for object and face detection to provide real-time visual perception. Voice command capability facilitated natural user interaction, while the dual 5-DOF robotic arms augmented the robot's capability for pick-and-place tasks.

From a design point of view, the robot was built effectively with laser-cut acrylic pieces and 3D-printed modules to achieve structural integrity and customization. The combination of the Raspberry Pi 5 with peripheral modules such as the Noir camera, voice recognition module, and ultrasonic sensors provided a compact yet robust control system.

In summary, this project not only met its technical goals but also set the stage for future enhancements, including improved emotion detection, obstacle avoidance with dynamic path adjustment, and cloud connectivity for remote operation. The assistive robot demonstrates the promise of low-cost, smart robotics to enhance quality of life in a broad range of applications—from smart homes to healthcare and beyond.

CHAPTER 7

REFERENCES

- [1] Ishiguro, Hiroshi, Tetsuo Ono, Michita Imai, and Takayuki Kanda. Development of an Interactive Humanoid Robot 'Robovie'—An Interdisciplinary Approach.,2001
- [2] Fong, Terrence, Illah Nourbakhsh, and Kerstin Dautenhahn. A Survey of Socially Interactive Robots.,2003
- [3] Ratsamee, Photchara, Yasushi Mae, Kazuto Kamiyama, Mitsuhiro Horade, Masaru Kojima, and Tatsuo Arai, Social Interactive Robot Navigation Based on Human Intention Analysis from Face Orientation and Human Path Prediction. ,2015
- [4] Mišeikis, Justinas, Pietro Caroni, Patricia Duchamp, Alina Gasser, Rastislav Marko, Nelija Mišeikienė, Frederik Zwilling, Charles de Castelbajac, Lucas Eicher, Michael Früh, and Hansruedi Früh, Lio-A Personal Robot Assistant for Human-Robot Interaction and Care Applications.,2020
- [5] Mariagrazia Costanzo, Rossana Smeriglio, and Santo Di Nuovo, ” New Technologies and Assistive Robotics for Elderly: A Review on Psychological Variables”,2024
- [6] Duerstock, Bradley S, Report on the Use of Assistive Robotics to Aid Persons with Disabilities,2024